

```
const express = require('express');
const crypto = require('crypto');
const app = express();
const PORT = 3000;

// Middleware para parsear JSON
app.use(express.json());

/**
 * Función auxiliar para validar parámetros
 */
const validateStrings = (req, res, keys) => {
  for (const key of keys) {
    if (req.body[key] === undefined || typeof req.body[key] !== 'string') {
      res.status(400).json({
        status: "error",
        error: `El parámetro '${key}' es requerido y debe ser una cadena.`
      });
      return false;
    }
  }
  return true;
};
```

PASO 1

```

PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3> npm init -y
Wrote to C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\package.json:

{
  "name": "practica3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",

```

PASO 2

```

PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3> npm install express crypto
npm warn deprecated crypto@1.0.1: This package is no longer supported. It's now a built-in Node module. If you've depended on crypto, you should switch to the one that's built-in.

added 66 packages, and audited 67 packages in 7s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

PASO 3

```

found 0 vulnerabilities
PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3> node server.js
at validateStrings (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\server.js:14:21)
at C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\server.js:27:10
at Layer.handleRequest (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\lib\layer.js:152:17)
at next (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\lib\route.js:157:13)
at Route.dispatch (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\lib\route.js:117:3)
at handle (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\index.js:435:11)
at Layer.handleRequest (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\lib\layer.js:152:17)
at C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\index.js:295:15
at processParams (C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica3\node_modules\router\index.js:582:12)

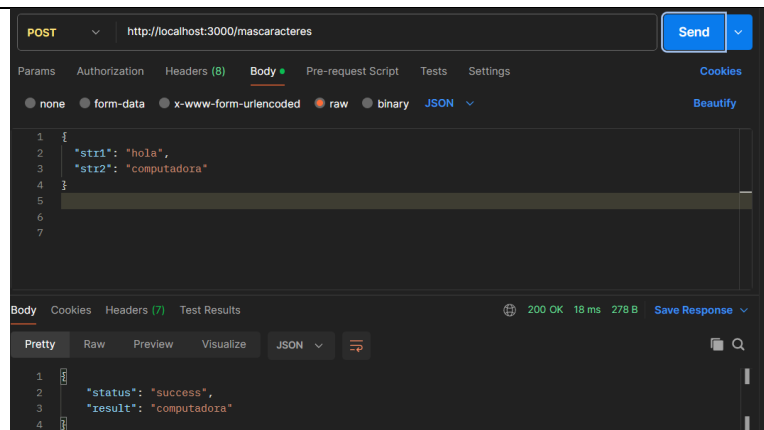
```

PASO 4***Realizamos el punto número i. Mas caracteres***

```

// i. mascaracteres
app.post('/mascaracteres'
, (req, res) => {
  if (!validateStrings(req,
res, ['str1', 'str2'])) return;
  const { str1, str2 } =
req.body;
  const result =
str2.length > str1.length ?
str2 : str1;
  res.json({ status:
"success", result });
});

```



PASO 5***Realizamos el punto número ii. Menos caracteres***

// ii. menoscaracteres

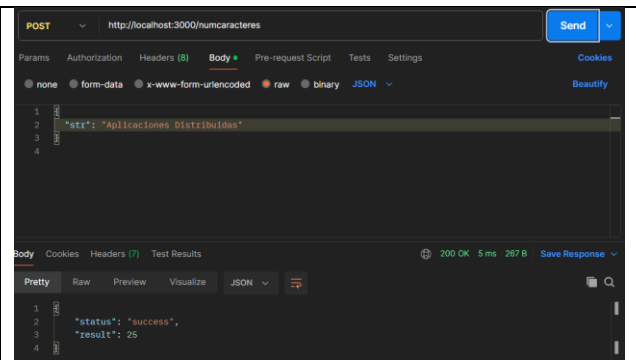
```
app.post('/menoscaracteres', (req, res) => {  
  if (!validateStrings(req, res, ['str1', 'str2'])) return;  
  const { str1, str2 } = req.body;  
  const result = str2.length < str1.length ? str2 : str1;  
  res.json({ status: "success", result });  
});
```



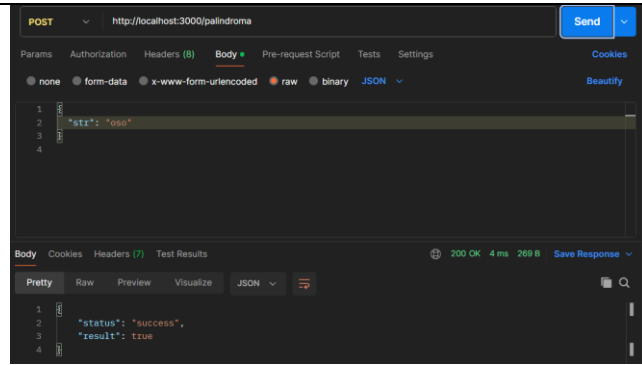
Si tienen la misma longitud manda la primera palabra

PASO 6***Realizamos el punto número iii. Número de caracteres***

```
// iii. numcaracteres  
app.post('/numcaracteres', (req, res)  
=> {  
  if (!validateStrings(req, res, ['str']))  
  return;  
  const { str } = req.body;  
  res.json({ status: "success", result:  
str.length });  
});
```

**PASO 7*****Realizamos el punto número iv. Palabra Palíndroma***

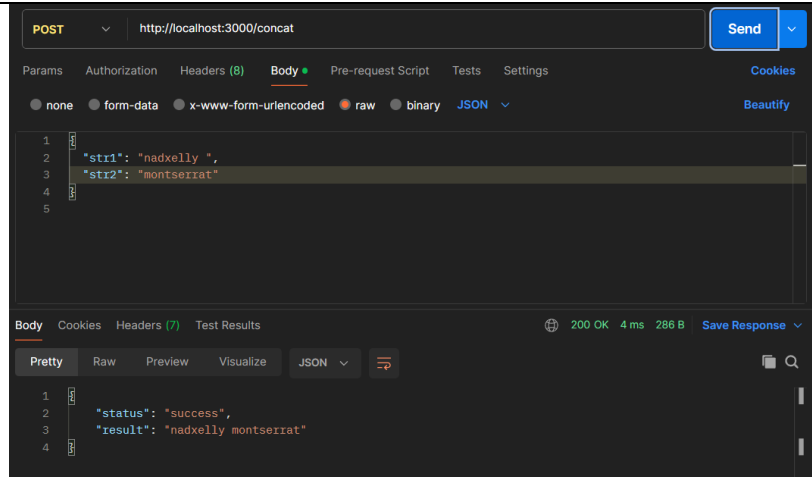
```
// iv. palindroma
app.post('/palindroma', (req, res)
=> {
  if (!validateStrings(req, res,
['str'])) return;
  const { str } = req.body;
  const cleanStr =
str.toLowerCase().replace(/[W_]/g
, "");
  const reversedStr =
cleanStr.split("").reverse().join("");
  res.json({ status: "success",
result: cleanStr === reversedStr });
});
```



PASO 8

Realizamos el punto número v. concat

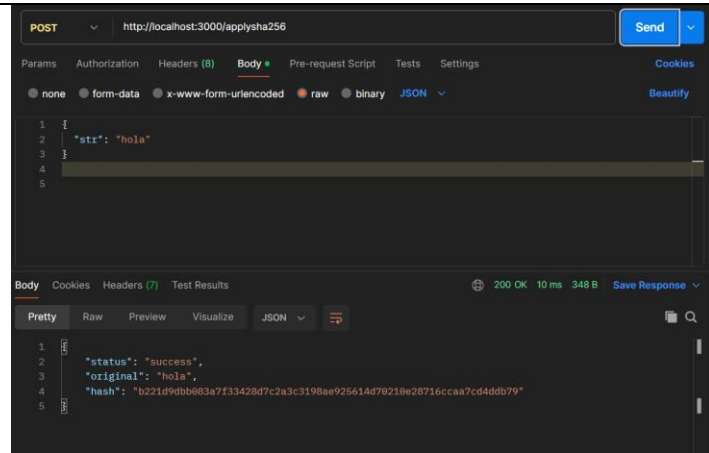
```
// v. concat
app.post('/concat',
(req, res) => {
  if
(validateStrings(req,
res, ['str1', 'str2']))
return;
  const { str1, str2 } =
req.body;
  res.json({ status:
"success", result: str1
+ str2 });
});
```



PASO 9

Realizamos el punto número vi. Aplysha256

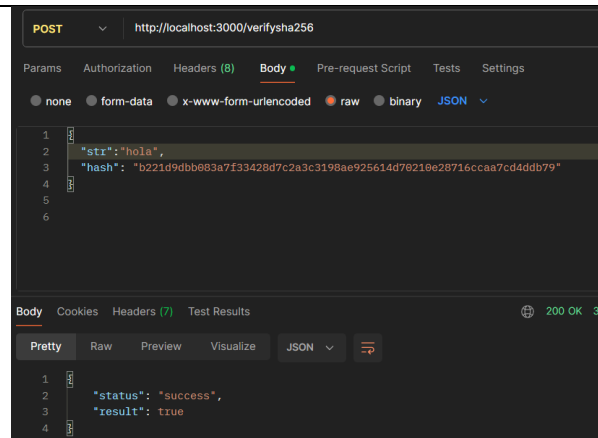
```
// vi. applysha256
app.post('/applysha256', (req, res) => {
  if (!validateStrings(req, res, ['str'])) return;
  const { str } = req.body;
  const hash =
    crypto.createHash('sha256').
    update(str).digest('hex');
  res.json({ status: "success",
    original: str, hash: hash });
});
```



PASO 10

Realizamos el punto número vii. Aplysha256

```
// vii. verifysha256
app.post('/verifysha256', (req, res) => {
  if (!validateStrings(req, res, ['str',
    'hash'])) return;
  const { str, hash } = req.body;
  const generatedHash =
    crypto.createHash('sha256').update(str
    ).digest('hex');
  res.json({ status: "success", result:
    generatedHash === hash });
});
```



PASO 11

```
app.listen(PORT, () => {
  console.log(` Servidor corriendo en http://localhost:${PORT} `);
});
```